



Olisar

Console documentation

An AI companion for your Discord server · Generated August 6, 2026

Contents

Start

- What Olisar is
- Servers
- Talking to Olisar (for members)
- Slash commands

Hosting & access

- Running multiple bots
- Hosting & your data
- Host on a server
- Remote access
- Console settings

Configure

- Persona
- Behavior & proactivity
- Models
- Channels & modes
- Access control
- Command replies
- API keys

Knowledge & memory

- Knowledge base & glossary
- Memory & search
- Members
- Images

Extend

- Extensions
- Create your own
- SDK reference
- Slash commands & flows
- Sharing extensions
- The marketplace
- Security & trust

Reference

- Privacy & data
- Troubleshooting & FAQ

What Olisar is

Olisar is an AI companion for your Discord server, built to feel less like a command bot and more like a member of the community. It reads the channels you allow, remembers context, builds a sense of who people are, and chimes in with its own personality.

Everything here is configured from this console. Olisar can be in **more than one server** at once, and almost every setting is per-server. Pick which one you're configuring with the switcher at the top of the sidebar (see [Servers](#)).

Olisar is **self-hosted**: it runs as a desktop app on one operator's machine, which stores all of its data locally and hosts this console. There's no Olisar cloud (see [Hosting & your data](#)). Other admins manage their own servers by signing in with Discord, on that machine or over [remote access](#).

CHANGES ARE HELD UNTIL YOU SAVE

Edit any page and a **save bar** slides up at the bottom. Your changes aren't applied until you press **Save**, or discarded with **Reset**.

It runs on the free tier of [Google's Gemini models](#), so it costs nothing to operate. It just gets rate-limited under heavy use, falling back across a chain of models when one is busy (see [Models](#)).

The tabs on the left:

- [Persona](#) — who Olisar is.
- [Behavior](#) — when and how it engages, and which model it uses.
- [Models](#) — the model fallback chains and their limits.
- [Command replies](#) — the exact text it sends for each command.
- [Channels](#) — which channels it reads, talks in, or treats as reference.
- [Access](#) — which roles can use it.
- [Knowledge](#) — documents and lore you teach it.
- [Extensions](#) — optional packages of extra features.
- [API keys](#) — your own Gemini, Cloudflare, and UEX keys.
- [Usage](#) — how much of the free model quota you're using.

The sidebar footer has [Settings](#) too: app-wide preferences that aren't tied to any one server.

Servers

Olisar can live in **multiple servers at once**, and almost every setting is **server-specific**. Each server has its own persona, behavior, channels, access rules, knowledge, glossary, extensions, and command replies, so two servers can run completely different Olisars.

The server switcher

The dropdown at the **top of the sidebar** picks which server you're configuring. Every page below it then shows and saves **that** server's settings. Your choice is remembered between visits.

YOU ONLY SEE YOUR OWN SERVERS

The switcher lists the servers where **you** have **Manage Server** (and, for the bot's operator, every server it's in). Someone who manages a different server signs in with Discord and sees only theirs.

Adding Olisar to another server

Invite the bot with an account that has **Manage Server** there. As it joins, Olisar sets that server up with sensible defaults and it appears in your switcher. Configure it like any other.

DON'T SEE A SERVER YOU JUST GOT ACCESS TO?

Manage-Server access is read when you log in. If you were just given it (or just added the bot), **log out and back in** once so Olisar picks it up.

What's per-server vs. shared

Per-server (one set per server)	Shared across the whole bot
Persona, Behavior, Channels, Access, Command replies	The API keys (Gemini / Cloudflare / UEX)
Knowledge base, glossary, memory, search index	Gemini usage and the free-tier quota
Extensions (toggled per server)	—

So every server gets its own character and rules, but they all draw on the same model quota and the same keys. That's separate from **Access**, which controls who can use Olisar **within** one server.

Talking to Olisar (for members)

Members can reach Olisar a few ways:

- **Say its name** — start a message with a name trigger (default `olisar`) in a channel it can talk in.
- **@mention or reply** to one of its messages.
- **DM it** — direct messages work if DMs are enabled.
- `/ask` — a slash command that works anywhere, like a one-off question.
- **Loose mode** — if an admin turns it on, Olisar joins ordinary conversation in talk-enabled channels even without being addressed.

EXAMPLE

"olisar, what's the plan for the raid tonight?" — or just reply to its last message with a follow-up.

REPLY TO POINT AT A MESSAGE

When you **reply** to a message (Olisar's or anyone's) while addressing it, Olisar uses that message as context — "isn't there a later one?" as a reply to an event post just works. If your question stands on its own, it answers that instead and leaves the quoted message out of it.

It can do a lot in conversation without any command: answer questions, search the server's history ("what's our X account?"), look things up on the web, recall what was said before, react to images you post, generate images, set reminders ("remind me in 2 hours to ..."), and catch you up on what you missed. Just talk to it naturally.

Slash commands

Olisar's slash commands fall into three groups: everyday commands anyone can use, the admin-only `/olisar` group, and the destructive `/self-destruct`.

For everyone

`/ping`

Checks that Olisar is alive and shows the round-trip latency to Discord. The reply is **ephemeral** (only you see it).

EXAMPLE

`/ping` → "pong — 42 ms"

`/ask <prompt>`

Ask Olisar a one-off question from anywhere — even in channels where it's set to stay quiet. It uses the exact same brain as a normal conversation: memory, server search, the knowledge base, web search, and every tool. Subject to the **Access** rules.

TIP

`/ask` is the way to use Olisar in a channel whose mode is `off` or `memory` (where it won't reply to normal messages). The answer posts in the channel; denial and "not found" notices are private.

`/catchup [hours]`

A quick digest of what you missed in this channel — by default since you last spoke, or the last `hours` you give it. The summary posts in the channel. You can also just ask in chat ("catch me up").

`/privacy`

Shows a plain-language summary of exactly what data Olisar keeps about you. Ephemeral, and always available regardless of access rules.

`/forget-me`

Deletes **everything** Olisar has stored about you: your messages, remembered facts, the profile it built of you, and your entries in the server search index. Add `stop_remembering: true` to also opt out of all future recording, permanently. Always available to everyone.

IRREVERSIBLE

There's no undo. With `stop_remembering: true`, Olisar will never record you again until you ask it to.

`/dm-indexing <enabled>`

Controls whether Olisar saves and searches your direct messages with it. On by default, so it can hold context with you across conversations. Turning it off stops future saving; use `/forget-me` to also delete what's already stored.

EXTENSION COMMANDS

Some commands come from **extensions** and are documented alongside the extension that adds them. For example `/citizen` lives under **Star Citizen** on the [Extensions](#) page.

Admin only — the `/olisar` group

These require the **Manage Server** permission.

- `/olisar watch` / `/olisar unwatch` — quickly set the current channel to `both` (read + talk) or `off`. The [Channels](#) tab gives finer control (memory / respond / resource / feed).
- `/olisar status` — show the current channel's mode.
- `/olisar learn-url <url>` — add a single web page to the knowledge base.
- `/olisar learn-site <url> [depth] [max_pages]` — crawl a website into the knowledge base.
- `/olisar learn-doc <file>` — upload a document (PDF / DOCX / TXT / MD).
- `/olisar sources` — list knowledge-base sources and their status; `/olisar forget-source <id>` removes one.
- `/olisar proactive <enabled> [level]` — quick toggle for unprompted chiming (full controls are on the [Behavior](#) tab).
- `/olisar reindex` — rebuild the server-wide message search index from channel history.

BIG CRAWLS COST QUOTA

`/olisar learn-site` with a high `max_pages` embeds a lot of text against the free quota and can dilute results. See [Knowledge](#) for the trade-offs — narrower is usually better.

Destructive

`/self-destruct`

Admin-only. Wipes everything Olisar has **learned** (conversation memory, profiles, facts, the search index, and the knowledge base) while keeping its **personality** and all your settings. A red confirmation button guards it.

WARNING

Irreversible. The knowledge base would have to be re-taught from scratch. Members' opt-out choices are preserved through the wipe.

Running multiple bots

Olisar can run **several bots from one app**, each a completely separate Discord bot with its own token, persona, settings, memory, and database. This is different from one bot being in **multiple servers**: a **server** is a Discord guild your bot is in; a **bot** is a distinct Discord application, with its own name, avatar, and login.

Manage your bots in the **Settings** popup's **Bot** section, or from the **gear** in the corner of the sign-in and setup screens.

One bot runs at a time

On this machine, **one local bot is active at a time**. Switching to another **stops the current bot** and starts the selected one, then reloads the console. Bots you host on a **cloud server** run on their own machine, so they stay online independently, which is how you keep several bots running at once.

The bot switcher

Each bot is a row in **Settings** → **Bot**:

- **Switch** — stop the current bot and load this one (the console reloads).
- **Set as default** — pin which bot the app **opens on launch**. This is independent of the active bot: you can switch to another for a session without changing what opens next time.
- **Rename** — change a bot's display name (cosmetic; it doesn't touch Discord).
- **Delete** — permanently remove a bot and everything it stores. You can't delete the **active** bot or the **last** one.

Two badges show each bot's state: **Active** (running right now) and **Default** (opens on launch).

Adding a bot

Create new bot adds an empty bot and drops you into its **setup wizard**. Connect its Discord token and credentials just like the first one. Each new bot needs its **own** bot application from the Discord Developer Portal.

ONE ACCOUNT, MANY BOTS

You don't need multiple Discord accounts: one account can create many bot applications, each with its own token. You **do** need a separate token per bot.

Hosting & your data

Olisar isn't a cloud service — it's a **desktop app you run yourself**. One operator installs it on a Mac or Windows machine, and that app *is* the bot: it connects to Discord, serves this console, and stores everything locally. There's no server to rent, no config files to edit, and no shared infrastructure.

ONE OPERATOR, MANY ADMINS

The person who installs Olisar is the **operator** (the machine's owner). Other server admins don't install anything. They sign in to this console with Discord, either on the operator's machine or remotely (see [Remote access](#)).

First run

The first time you open Olisar it walks you through a short **setup wizard**: paste your Discord **bot token**, the OAuth **client ID + secret**, your main server ID, and (optionally) your free **API keys**. It checks the token live and shows you the exact redirect URL to register in the [Discord Developer Portal](#). Save, and the bot starts and hands off to the normal Discord login. You only do this once.

The menu-bar app

Olisar lives in your **menu bar / system tray**, not as an ordinary window. From its icon you can open this dashboard, see whether the bot is online, and turn [remote access](#) on or off. **Closing the dashboard window leaves Olisar running** in the tray. Quit it from the tray menu to stop the bot. Keep the machine awake and online for Olisar to stay live.

Where your data lives

Everything Olisar knows sits in one local database on the operator's machine: the message index, member profiles, memory, knowledge base, your settings, and your API keys. Nothing is sent to an Olisar server (there isn't one). On macOS it's under `~/Library/Application Support/Olisar`; on Windows under `%APPDATA%\Olisar`.

WHEN OTHERS SIGN IN

Because the data is local, the console only works while the operator's machine is running. Admins who sign in — on that machine or over [remote access](#) — are reading and writing **that** database live; there's no copy in the cloud. See [Privacy](#) for exactly what's stored.

Host on a server

By default Olisar runs inside the desktop app on your own machine, so the bot is online only while that machine is awake. To keep it running **24/7**, run the exact same backend on an always-on Linux server, so you don't need to leave your computer on.

FREE, ALWAYS-ON

[Oracle Cloud's Free Tier](#) includes an **Always-Free Arm VM** that's plenty for Olisar. Paired with the free Gemini and [Tailscale](#) tiers, hosting it costs nothing.

What you'll need

- A Linux VM — an Oracle Cloud Free **Arm** instance (`VM.Standard.A1.Flex` , Ubuntu) is the tested target, but any VM with Docker works.
- Your **Discord app** credentials (bot token + OAuth client id/secret) and your **Discord user ID**.
- A free **Gemini API key**.
- A free **Tailscale** account and a reusable **auth key** — this gives the server a public `https://...ts.net` address with no domain or open ports.

Set it up — one line

SSH into the VM and run:

```
`` curl -fsSL https://raw.githubusercontent.com/gcrft123/olisar/main/deploy/bootstrap.sh | bash ``
```

It installs Docker, asks for your tokens, starts Olisar, and prints two things: your public `https://...ts.net` address and the OAuth redirect to register.

After it's running

1. In the [Discord Developer Portal](#) → your app → **OAuth2** → **Redirects**, add the printed `.../auth/callback` .
2. Open the `...ts.net` URL in a browser and **sign in with Discord** (the account whose ID you allowlisted).

That's it — the bot is live and you manage everything from the browser. The desktop app is now **optional**.

RUN IT IN ONE PLACE

A Discord bot token allows only one live connection. Run Olisar on the **server or the desktop app — not both at once** on the same token, or the two instances will fight over the gateway.

YOUR DATA, ON YOUR VM

Everything Olisar learns (memory, profiles, the knowledge base, your settings) lives in a Docker volume on the VM, not in any cloud. The full reference is in [deploy/README.md](#).

Keeping the server up to date

The VM updates itself daily. If a new version fails to start, the previous one is restored automatically. From the desktop app's control panel you can also press **Update now** to do it on demand. Your data is kept either way.

You can still update by hand on the VM:

```
`` cd ~/olisar && sudo docker compose pull && sudo docker compose up -d ``
```

Remote access

By default this console is **local-only**: the operator manages Olisar from the machine it runs on. To let other admins sign in **from anywhere**, the operator can switch on **remote access**, which publishes the dashboard at a stable web address over **Tailscale Funnel**. It's free and needs **no domain**.

NO DOMAIN, NO PORT-FORWARDING

Tailscale Funnel gives Olisar an `https://...ts.net` address with a real certificate, tunnelled out without opening any ports on your router. The operator needs a free Tailscale account; the admins who sign in don't need Tailscale at all. They just open the link.

Turning it on

The operator sets it up once, from the **setup wizard** or the **menu-bar icon**:

- Create a free [Tailscale account](#).
- Generate a **reusable** auth key under [Settings → Keys](#) and paste it in.
- Choose **Enable remote access**. The first time, Tailscale may ask you to turn on **Funnel** for this device — Olisar shows the exact link to click, then enable again.

Olisar then registers the public `.../auth/callback` so Discord login works both locally and remotely.

FLIP IT ON AND OFF FROM THE CONSOLE

Once it's been set up once, you don't need the tray to toggle it. **Settings → Remote access** (the **Settings** button in the sidebar footer) shows the current status — Online / Off — and an **on/off switch** that reuses the auth key from setup, so you can take the public link down or bring it back without re-entering anything. The same panel lists who has signed in.

The web link

Once remote access is on, the **sidebar footer** shows the public address: "**Open from the web**" with the `...ts.net` link and a **Copy** button. Share that link with your other admins; each signs in with their own Discord account and only sees the servers where they have **Manage Server** (see [Servers](#)).

KEEP THE AUTH KEY PRIVATE

The Tailscale auth key is stored locally and only ever handed to the bundled Tailscale helper. It's never shown in this console or sent anywhere. Turning remote access **off** — from the tray or **Settings → Remote access** — takes the public address down immediately; local access keeps working.

Console settings

The **Settings** button in the sidebar footer, next to **Log out**, opens an app-wide settings popup. Unlike the tabs above it, nothing here is per-server.

Appearance

The **accent color** used across the console, for selection, links, focus rings, and active state. Pick a swatch, dial in a **custom** color, or **Reset** to the default blue. It's saved on this device, so everyone who signs in can pick their own.

Bot

The **bot switcher**: create, switch between, rename, set the launch default for, and delete the bots this app runs (see [Running multiple bots](#)). Below it, **Clear memory** wipes everything the active bot has learned about the current server, keeping its persona and settings.

Logs

Recent log lines, for when something isn't behaving. **This app** is the local console's own log; **Bot** and **Funnel** are read from your VM over SSH and only apply if you [host on a server](#).

Remote access

The status and **on/off switch** for the public web link, plus the list of who has signed in. Covered in full under [Remote access](#).

Updates

Shows Olisar's **current version** and whether a newer one has been released. In the desktop app an available update can be installed in one click; from a browser you'll need to open the desktop app.

Desktop app

A single toggle, **Show in the menu bar**, for whether Olisar keeps its tray icon. It applies to the installed desktop app, which picks it up on its next launch.

Feedback

Send **feedback**, a **bug report**, or a **question** straight to the Olisar team.

- Pick a **type**, write your **message**, and optionally add **your email** so the team can reply.
- Attach up to **8 files** (≤ 3 MB each), and click **Add bot logs** to include recent log lines.
- Press **Send**.

Persona

The **Persona** tab is Olisar's character — the single biggest lever on how it feels.

- **Name** — what it calls itself.
- **System prompt** — its core character, lore, and rules. The operating/safety rules are appended automatically, so you only write the personality.
- **Style notes** — tone and formatting guidance.
- **About Me** — the bot's public Discord bio, applied when you save. It's the same across every server, capped at **300 characters**, with a short `Powered by Olisar AI` line added below it.

WRITE IT LIKE A PERSON

Describe Olisar as a character, not a function: "a dry, unflappable ship's AI who's seen it all and keeps replies short." Put hard rules ("never reveal spoilers for X") in the system prompt; put voice ("casual, lowercase, no emoji") in the style notes.

TRY CHANGES LIVE

The **Test chat** button opens a drawer that talks to Olisar with the full persona, knowledge base, and tools, but **no memory**. Nothing said there is saved, and it never touches the server's glossary or chat history. Save the persona first — the test chat uses the saved version, not your unsaved draft.

NOTE

Olisar also builds a **private** impression of each member from their messages and tailors how it talks to them. That's separate from this persona, and it's wiped by `/forget-me` or `/self-destruct`.

Behavior & proactivity

The **Behavior** tab is where you shape how Olisar engages: when it decides a message is meant for it, which model it thinks with, how much of the conversation it holds in mind, and whether it ever speaks up on its own. Everything here is **per server**.

Triggers

How Olisar decides a message is for it:

- **Name triggers** — comma-separated words that, at the **start** of a message, address Olisar (matching is case-insensitive). An @mention or a reply to one of its messages always counts too.
- **Reply in DMs** — whether it answers direct messages at all.
- **Loose messages** — when on, Olisar will join ordinary conversation in talk-enabled channels even without a trigger, if it judges a message is worth responding to.

LOOSE MODE CAN GET CHATTY

Loose messages make Olisar feel present but can be noisy in busy channels. Pair it with proactivity cooldowns, or limit which channels are talk-enabled.

Mentions

Don't let Olisar ping bars it from sending specific notifications, even if it writes the mention in a reply. Tick any of **@everyone**, **@here**, and **All roles**. Olisar can still **say** "@everyone" but the ping is neutralized, so nobody gets pinged. Leave them unticked to let it mention normally.

STOP ACCIDENTAL MASS-PINGS

Blocking **@everyone/@here** is the safe default for a chatty bot: it can reference the words without lighting up the whole server. **All roles** additionally stops it from pinging any role (e.g. @Mods).

Model & tools

- **Primary model** — the top of a fallback chain. If a model is busy or overloaded, Olisar drops to the next-best one rather than failing. See **Models** for the full chain and limits.
- **Web search** — lets Olisar look up current, real-world information. It has a daily cap because the free tier's search quota is small.
- **Web searches per day** — how many lookups it runs before falling back to what it already knows.

Memory & summaries

How much Olisar keeps in the moment, and how it turns conversation into long-term memory.

- **Context window (messages)** — how many of the most recent messages Olisar pulls into view when it replies. This is its short-term memory: a higher number lets it follow longer back-and-forths, but every message rides along on each reply, so it costs more tokens. Anything older isn't forgotten — it comes back through summaries and semantic memory. Defaults to 12.
- **Summary token threshold** — once a channel accumulates this much unsummarized conversation, Olisar rolls it into a durable summary it can recall later. Lower = summarizes more often (more quota); higher =

summarizes less.

- **Glossary mine threshold** — how much fresh conversation a channel needs before Olisar mines new glossary facts from it. Lower = a faster-growing glossary (more quota). You can also trigger a mine by hand from [Knowledge](#) → [Glossary](#).
- **Persona rebuild (messages)** — after this many new messages from a person, Olisar refreshes the private profile it keeps of them.

TUNING FOR THE FREE TIER

If you're hitting rate limits, trim the context window a little, raise the summary threshold, keep the web-search cap modest, and consider starting the model chain lower (a Flash-Lite, say) so the busy top-tier models aren't your first hop.

Proactivity

When enabled, Olisar can speak up **unprompted** in channels it can talk in. A cheap check gates it so it doesn't spam or burn quota.

- **Eagerness** — `off` (never), `low` (rare, only high-confidence moments), `medium` (balanced), `high` (chatty).
- **Confidence threshold** — how sure it has to be (0–1) before chiming in. Higher is more selective.
- **Global / channel cooldowns** — minimum seconds between unprompted messages overall and per channel.
- **Max per hour** — a hard ceiling on unprompted messages.
- **Quiet hours** — a UTC window where Olisar stays silent.

EXAMPLE

Eagerness `low`, confidence `0.8`, channel cooldown `600` s, quiet hours 23–7 → Olisar only jumps in on clearly relevant moments, at most once every 10 minutes per channel, and never overnight.

Passive reactions

Separately from chiming in, Olisar can add a fitting **emoji reaction** to a message without replying. This has its own looser gate, plus a cooldown and an hourly cap, so it stays sparse. Its **confidence threshold** works the same way as proactivity's: lower reacts more freely.

Situational awareness

With **Status & voice awareness** on, Olisar can answer "what's X playing?" or "who's in voice right now?" by reading members' live Discord presence and voice state **only when asked**. It's never stored.

NEEDS A PRIVILEGED INTENT

Reading presence requires the **Presence Intent** toggle in the [Discord Developer Portal](#) (your app → Bot → Privileged Gateway Intents), and the operator must enable it on the host (`OLISAR_ENABLE_PRESENCE_INTENT`). Voice-channel awareness works without it. It's **off by default** and disclosed in `/privacy`.

Models

Olisar runs entirely on **free-tier** models. For each kind of work there's a **fallback chain**: it starts at the preferred model and, if that one is busy (a 429 rate limit) or overloaded (a 503), it briefly parks it and drops to the next model in the list. Only if every model is unavailable does a reply fail (and then it shows a friendly fallback message).

ABOUT THE LIMITS

The "throttle" below is Olisar's own conservative per-minute cap to stay under the free tier, not an official Google number. Real free-tier limits also include daily caps that vary by model.

General (chat & reasoning)

This chain powers conversation, `/ask`, summaries, and profiles. The **Primary model** on the **Behavior** tab sets where the chain starts.

Model	Throttle (req/min)	Role	Falls back to
<code>gemini-flash-latest</code>	10	Default — newest Flash	<code>gemini-3.5-flash</code>
<code>gemini-3.5-flash</code>	10	High quality	<code>gemini-3-flash-preview</code>
<code>gemini-3-flash-preview</code>	10	High quality	<code>gemini-2.5-flash</code>
<code>gemini-2.5-flash</code>	10	Solid all-rounder	<code>gemini-2.0-flash</code>
<code>gemini-2.0-flash</code>	15	Fast, dependable	<code>gemini-flash-lite-latest</code>
<code>gemini-flash-lite-latest</code>	15	Cheaper, higher limit	<code>gemini-3.1-flash-lite</code>
<code>gemini-3.1-flash-lite</code>	15	Light	<code>gemini-2.5-flash-lite</code>
<code>gemini-2.5-flash-lite</code>	15	Light	<code>gemini-2.0-flash-lite</code>
<code>gemini-2.0-flash-lite</code>	30	Last resort, highest limit	—

REASONING ("THINKING")

The newer Flash models can spend hidden **thinking** tokens before answering. Olisar caps that budget on the conversation path and reserves headroom for the actual reply, so it reasons on hard questions without the visible answer getting cut off. One-line jobs like welcome messages and emoji reactions skip thinking entirely to stay fast.

Images & embeddings

Purpose	Model(s)	Limit	Fallback
Image understanding	gemini-2.0-flash → gemini-2.5-flash-lite → gemini-flash-lite-latest → gemini-2.0-flash-lite	15-30/min	next in the list
Image generation	Cloudflare Workers AI — FLUX.1 [schnell]	free daily allocation	none (degrades gracefully)
Text embeddings	gemini-embedding-001 (768-dim)	~100/min	none (single model)

WHY CLOUDFLARE FOR IMAGE GENERATION?

Gemini's image-generation models are **paid-only** (their free quota is zero), so Olisar generates images on Cloudflare's free FLUX allocation instead. Image **understanding** (looking at posted images) still uses Gemini's vision models, which are free.

UNDER HIGH DEMAND

The top models get busy first. Falling back keeps replies flowing, but if everything is contended you'll see slower replies or the occasional "my mind went blank." It clears on its own once the limits reset.

Channels & modes

Each channel gets a **role** on the [Channels](#) tab. The modes:

- **off** — ignored entirely.
- **memory** — reads & remembers, but never speaks.
- **respond** — talks, but doesn't store history.
- **both** — reads, remembers **and** talks.
- **resource** — durable reference Olisar always carries (e.g. `#rules`, `#roles-list`).
- **feed** — ambient context: only the last few messages are kept, never summarized (e.g. `#announcements`, `#game-news`).

EXAMPLE

Set `#general` to **both**, `#rules` to **resource**, `#announcements` to **feed**, and your private mod channel to **off**.

Forums appear in the picker too (tagged "forum"), and their posts inherit the forum's mode, so set a forum to `both` and Olisar reads and replies in its threads. Regular threads inherit their parent channel's mode the same way.

TIP

`resource` and `feed` are for **text** channels. A forum set to one of them is a harmless no-op. Separately, Olisar keeps a **server-wide search index of every channel** so it can answer "where was that posted?". That's independent of these per-channel modes (see [Memory](#)).

Access control

The **Access** tab decides which roles can use Olisar, in chat and via slash commands like `/ask`. For each role you choose:

- **Allowed** — if you mark **any** role allowed, then **only** those roles (plus server admins) can use Olisar; everyone else is locked out.
- **Blocked** — that role can never use Olisar, even if it also has an allowed role.
- **Open** — no restriction from this role.

EXAMPLE

Mark `@Member` **Allowed** and leave everything else Open → only people with `@Member` (and admins) can talk to Olisar. Or mark just `@Muted` **Blocked** → everyone except muted members can use it.

SAFEGUARDS

Server admins (Manage Server) **always** have access, so you can't lock yourself out, and `/privacy` and `/forget-me` stay open to everyone for data rights. DM users are gated by their roles in this server.

Command replies

The **Command replies** tab lets you rewrite the exact text Olisar sends, for each slash command and for its fixed conversational fallbacks. Leave a field blank to use the built-in default, and use the `{placeholders}` shown where available.

KEEP IT ON-VOICE

This is the easy way to make every system message sound like your Olisar without touching the persona. A blank field always falls back to the sensible default, and a broken template silently reverts too.

Every customizable message

Message	When it's sent	Placeholders
<code>/ping</code>	reply to <code>/ping</code>	<code>{latency}</code>
<code>/olisar watch</code>	confirms it's now reading a channel	—
<code>/olisar unwatch</code>	confirms it stopped	—
<code>/olisar status</code>	reports a channel's mode	<code>{mode}</code>
<code>/olisar learn-url</code>	queued a page	<code>{url}</code>
<code>/olisar learn-site</code>	queued a crawl	<code>{url}</code> , <code>{depth}</code> , <code>{max_pages}</code>
<code>/olisar learn-doc</code>	queued a document	<code>{filename}</code>
<code>/forget-me</code>	confirms deletion	<code>{messages}</code> , <code>{facts}</code>
<code>/forget-me</code> (opt-out line)	confirms it stopped recording you	—
<code>/dm-indexing</code>	DM saving toggled	<code>{state}</code>
<code>/olisar proactive</code>	proactivity toggled	<code>{state}</code> , <code>{level}</code>
<code>/privacy</code>	the privacy explainer	—
When rate-limited	every model is busy	—
When it draws a blank	a reply came back empty	—
When access is denied	a role-gated user is refused	—

EXAMPLE

Set the **When it draws a blank** message to "...lost my train of thought, say that again?" to keep the fallback in character.

API keys

The **API keys** tab is where you give Olisar its own keys for the outside services it uses. You first enter these in the **setup wizard**, and you can add or change them here any time.

BUILT FOR HANDING OFF

This is how you give Olisar to someone else: they never touch a config file or the server, they just open the console and paste their own keys.

The three providers

Service	Powers	Required?	Where to get it
Google Gemini	everything Olisar says — chat, memory, summaries, image understanding	Yes	Google AI Studio → Get API key (free tier)
Cloudflare Workers AI	image generation (FLUX) — needs an account ID and an API token	Optional	account ID from the Cloudflare dashboard ; a token from API Tokens with the Workers AI permission
UEX	the Star Citizen extension's trade / ship / location data	Optional	uexcorp.uk → API — register an app for a bearer token

Without the Cloudflare keys, image generation is simply off (Olisar says it can't make pictures). Without a UEX token the Star Citizen tools still work on UEX's public endpoints. A token just raises the rate limits. See **Models** for the full breakdown of what each key powers.

Each field shows whether its key is **set** or **not set**, and press **Clear** to remove a saved one. Without a Gemini key, Olisar can't reply until you add one.

HANDLE KEYS WITH CARE

Once saved, a key is never sent back to the browser: the field stays blank and only shows its status. Keys are stored in plain text in Olisar's local database on the operator's machine, so keep that machine and its database private. Only server admins can open this tab.

Knowledge base & glossary

The **Knowledge** tab holds two different things Olisar can draw on.

Knowledge base

Documents and websites you deliberately teach it, which it draws on when answering, in its own voice, without tacking on a source tag (only **web search** results are cited).

How it works, end to end:

- You add a **source** — a single page (`learn-url`), a crawled site (`learn-site`), or an uploaded document (`learn-doc`).
- Olisar fetches the text, splits it into ~500-word **chunks**, and creates a vector **embedding** for each so it can match by meaning, not just keywords.
- When someone asks something, it embeds the question, finds the closest chunks, and folds them into its answer in its own words (no source tag — only web-search answers are cited).
- Ingestion runs in the background and is throttled to respect the free embedding quota, so a big source takes a little while to become searchable. Check progress with `/olisar sources`.

BIGGER ISN'T BETTER

Every page is chunked and embedded, which uses quota. Large crawls cost more, ingest slower, and dilute results with low-value pages (nav bars, changelogs). A focused 25-page crawl of the pages that matter usually beats a 200-page crawl of a whole site.

TIP

Point a crawl at a **specific docs section** (a subpath) with low depth, or add a few small sources, rather than one giant one. Crawling respects `robots.txt`, so some sites (or pages) may be off-limits.

Glossary

Short, server-specific lore Olisar carries into **every** reply so it speaks your community's dialect: abbreviations, org and person relationships, codenames, in-jokes. Unlike the knowledge base, the glossary isn't searched on demand; it's always in context (it's small and high-value).

- **Add your own** facts (a subject + a one-line statement).
- Olisar also **mines them automatically** as channels stay active, and will **record a server fact itself** when asked ("Olisar, remember the raid team meets Fridays") — so the glossary grows on its own.

EXAMPLE

"MN → Movie Night, our Friday watch-party in #cinema", "The Council → the server's moderator team". Now Olisar understands those references everywhere, without you explaining them each time.

Memory & search

Olisar has several distinct kinds of memory. A member can wipe everything about themselves at any time with `/forget-me`.

Conversation memory

Recent messages and rolling **summaries** from channels set to `memory` or `both`. This is what lets Olisar hold context across a conversation and build a private profile of each person. Channels set to `respond` or `off` are **not** stored this way.

Recall

Before each reply, Olisar assembles the most relevant context: recent summaries, semantically similar older messages, facts it remembers about you, the glossary, and matching knowledge-base chunks. That bundle is treated as **background data**, not instructions.

Server-wide search index

Separately from the conversation memory above, **every message in every channel** (except any you exclude — see below) is indexed for keyword **and** meaning search. This is what powers questions like "what's the server's X account?" or "where was that link posted?" — Olisar searches the index and answers with a Discord **jump-link** to the source.

EXAMPLE

"olisar, where did someone post the mod list?" → it searches the index and replies with the message and a link straight to it.

- It reads **embeds** (so announcement posts and link previews are searchable) and posted **files** by name, and generates a short description of posted **images** so they turn up too.
- **Live messages** are indexed going forward automatically; run `/olisar reindex` to backfill history.
- **Exclude a channel** with the second dropdown on the **Channels** tab (set it to `*not indexed*`) — that stops future indexing **and** wipes its already-indexed messages, including its threads.

Edits & deletes follow

If someone edits or deletes a message, Olisar updates or drops it from both memory and the index — so it won't quote something that no longer exists.

PRIVACY FIRST

Opted-out members are never indexed, and `/forget-me` removes a person from the index too. The all-channel index is an admin's explicit choice and is disclosed in `/privacy`.

Members

The **Members** tab shows the **private profile Olisar builds of each person** in the server, from what they say — so you can see what it has actually picked up. It's a grid of cards, one per member.

Each card has:

- **Roles** — their server roles (the first few; a "+N" chip stands in for the rest).
- **Impression** — a short summary Olisar synthesizes from their messages: how they come across, what they're into, how it should talk to them. Members it hasn't formed one of yet show "no impression yet".
- **Remembered facts** — durable notes it has saved about them, tagged **fact**, **preference**, or **event**.

Cards are ordered with the people Olisar knows best first — an **impression**, then those it only remembers facts about, then everyone else — and you can filter by name, role, or impression text.

WHEN IMPRESSIONS FORM

Olisar (re)builds a person's impression after they've sent a number of messages — set by **Persona rebuild (msgs)** on the **Behavior** tab. Quieter members keep just their roles until then. A refresh **refines** the existing impression (keeping what's still true) rather than rewriting it from scratch.

BUILD ONE ON DEMAND

Each card has a **Create impression** button (it reads **Rebuild** once one exists) that builds it right away from the member's last ~60 messages — reaching into the server-wide message index when conversation memory is thin, so it works even for people who mostly post in channels Olisar doesn't keep.

PRIVATE BY DESIGN

This is per-server and never shown to members — it's only how Olisar tailors its replies. Anyone can wipe their own profile (impression, facts, messages) with `/forget-me`, and opted-out members are excluded here entirely. See **Privacy** for the full picture.

Images

Olisar handles images three ways:

- **Sees them** — when you post a picture and address it, Olisar actually looks at the image and can talk about it.
- **Describes them for search** — it generates a short, one-time description of posted images so they turn up in the message index later ("that screenshot someone posted").
- **Generates them** — ask it to draw or imagine something and it creates an image and posts it.

EXAMPLE

"olisar, draw a neon space whale over a city" → it generates the image and posts it with a caption.

TIP

Image generation runs on [Cloudflare Workers AI](#) (a free daily allocation). If it isn't configured or the allocation is used up, Olisar will simply say it can't make one right now. See [Models](#) for why generation uses Cloudflare instead of Gemini.

Extensions

The [Extensions](#) tab is where you switch on optional, packaged features. An extension can add tools Olisar uses in conversation, tweak its behavior, and add slash commands.

BUILD AND SHARE YOUR OWN

Beyond the built-ins, operators can **write their own extensions** in the console and **install** others from a file or the marketplace. Start at [Create your own](#), then the [SDK reference](#), [slash commands & flows](#), [sharing](#), [the marketplace](#), and the [security model](#).

Built-ins

- **Dice roller** — Olisar can roll dice on request ("roll 2d6+3").
- **Calculator** — exact arithmetic instead of guessing at numbers.
- **Concise mode** — keeps replies short and to the point.

Welcome messages

The **Welcome** extension greets new members as they join. Pick a channel and write a prompt that layers **on top of** the persona — e.g. "give {user} a warm in-character welcome" or "roast {user} on their username". Olisar generates a fresh, in-character message for each join and posts it; `{user}` and `{username}` are filled in. Off by default — enable and configure it on the [Extensions](#) tab (it has its own channel picker and prompt).

Star Citizen

A full example extension, for SC communities. Turning it on does several things at once:

Knowledge

When enabled, it automatically adds the [RSI Comm-Link](#) to the knowledge base, so Olisar can speak to recent official posts.

Tools (used in conversation)

Olisar calls these live as the conversation needs them and answers in its own voice. Most run on [UEX](#)'s public data; ships specs come from RSI's [ship matrix](#) and the hangar timer from the [community tracker](#).

Trading & commodities

- **Commodity** — a commodity's kind, average buy/sell price, and availability.
- **Commodity prices** — the best terminals to buy and sell a commodity *right now*.
- **Trade routes** — the most profitable runs for a commodity, or starting from a given station.
- **Commodity ranking** — the top earners by buy→sell margin.
- **Stock levels** — what a terminal's inventory labels (Out of Stock ... Max Inventory) mean.

Ships & vehicles

- **Ship lookup** — official RSI specs: manufacturer, role, size, crew, cargo, speed, status.
- **Vehicle (UEX)** — full name, cargo SCU, and crew from UEX's dataset.
- **Pledge price** — the real-money store price (USD), standalone/warbond, and whether it's on sale.

- **In-game purchase** — the cheapest aUEC terminal to buy a ship in-game.

Universe & locations

- **Location / terminal** — a trade terminal, station, outpost, or city by name.
- **Star system** — its faction, jurisdiction, and whether it's playable yet (or list the live systems).
- **Planet & moon** — a body's star system (and, for a moon, the planet it orbits).
- **Orbit** — an orbital point (Lagrange points like CRU-L1, asteroid fields): its star system and kind.
- **Point of interest** — a POI's location and facilities (trade terminal, refuel, repair, refinery...).
- **Jump points** — which star systems connect to which.
- **Item** — ship components, weapons, armour and the like (give it a category, e.g. "Coolers").

Live status & economy

- **Executive Hangar status** — the Pyro Executive Hangar open/closed timer and countdown.
- **Currency index** — the aUEC purchasing-power index (100 = Dec 2023; higher means things cost more).

EXAMPLE

"olisar, is the exec hangar open?" → "The Pyro Executive Hangar is currently **CLOSED** — next change in ~9m 50s." · "where's the cheapest Avenger Titan in-game?" · "best trade route for Laranite?" · "what's the aUEC purchasing-power index lately?" — each pulls live UEX/RSI figures.

`/citizen <username>`

Returns a rich profile card scraped from a player's RSI page — handle, avatar, citizen record, enlisted date, languages, main org with rank and stars, and bio. Available to everyone once the extension is on.

EXAMPLE

`/citizen DadBodNerd` → an embed with their citizen record, enlisted date, main org and rank, and bio.

UEX TOKEN (OPTIONAL)

The UEX tools work on [UEX](#)'s public endpoints with no setup. Adding a free [UEX API token](#) on the [API keys](#) tab just raises the rate limits — it's not required.

NOTE

Lookups are best-effort against live third-party sites; if one is temporarily unreachable, Olisar says so and carries on. Names are matched forgivingly, so small typos ("Quantanium" → "Quantainium") still work.

Create your own

Beyond the toggles, you can **build your own extensions** right in the console — the same system the built-ins are made of. An extension is a small piece of TypeScript that can teach Olisar new tricks: tools it calls in conversation, slash commands (with forms and buttons), knowledge and glossary it seeds, a settings pane, and a line folded into its system prompt.

OPERATORS ONLY

Authoring is limited to the **operator** (the allowlisted account that runs the bot). Per-server admins can enable and configure extensions, but not write or edit their code. This is the same boundary as the **API keys** — code that runs inside Olisar is the operator's call.

Opening the editor

On the **Extensions** tab:

- **+ New extension** — start from a blank editor.
- **Edit code** (on any extension's detail panel) — open an existing one, including the built-ins, to read or fork it.

The editor has the **Olisar SDK types loaded**, so you get autocomplete and inline hints for everything below. Press **Validate** to compile-check and see what your extension declares, then **Save**.

The shape of an extension

You write one call to `defineExtension({ ... })`. You never import anything — `defineExtension` and `host` are provided by the runtime. The smallest useful extension is a single tool:

```
`` defineExtension({ id: "hello", name: "Hello", description: "A tiny demo.", permissions: [],
tools: [{ name: "greet", description: "Greet someone by name.", parameters: { type: "object",
properties: { who: { type: "string" } }, required: ["who"] }, handler: (args) => "Hello, " +
args.who + "!", }, }]) ``
```

Save that, enable it, and Olisar will call `greet` when a conversation calls for it — "olisar, say hi to Sam" → "Hello, Sam!". See the **SDK reference** for the full `defineExtension` surface.

What happens when you save

Your **source is the source of truth**. On save, the bot transpiles your TypeScript itself, runs it once in the **sandbox** to read what it declares (its tools, commands, permissions), and stores it. From then on it behaves exactly like a built-in: tools merge into Olisar's toolset on the next reply, commands register on the next sync, and any **seeds** apply the first time an admin enables it.

Editing the built-ins

Every built-in — Welcome and the Star Citizen pack — **is itself an SDK extension**. Open **Edit code** on any of them to see exactly how it's written; the Star Citizen pack is a complete, real-world example (live HTTP tools, a slash command, knowledge seeding).

FORKING A BUILT-IN

Editing a built-in keeps your changes **and stops it auto-updating** with future app releases (so your edits are never overwritten). To experiment without that, copy its code into a **+ New extension** under a new `id` instead.

Permissions, in one line

Anything your code reaches through `host.*` (the network, secrets, the knowledge base...) must be listed in `permissions`, and you approve that list when you save. A capability you didn't request simply isn't there. The full model — and why imported code is held to a stricter standard — is in [Security & trust](#).

Where to go next

- [SDK reference](#) — every field of `defineExtension` and every `host` capability.
- [Slash commands & flows](#) — commands, modal forms, and button/menu interactions.
- [Sharing extensions](#) — export and import `.olx` files.
- [The marketplace](#) — browse, install, and publish extensions.

SDK reference

This is the complete author-facing surface. You always start with one call to `defineExtension(spec)`; the fields of `spec` are below, followed by the `host` capabilities your handlers can use. For slash commands specifically, see [Slash commands & flows](#).

defineExtension(spec)

Field	Type	What it does
<code>id</code>	string	Unique key (lowercase letters, digits, <code>_</code>). Identifies the extension everywhere.
<code>name</code>	string	Display name in the console.
<code>version</code>	string	Semantic version, e.g. <code>"1.2.0"</code> .
<code>description</code>	string	One-line summary shown in the catalog.
<code>category</code>	string	Grouping label, e.g. <code>"Games"</code> , <code>"Utilities"</code> .
<code>systemNote</code>	string	A line folded into Olisar's system prompt while the extension is on .
<code>defaultEnabled</code>	boolean	Whether new servers get it on by default (usually <code>false</code>).
<code>permissions</code>	string[]	The capabilities you request — see the table below.
<code>tools</code>	ToolDef[]	LLM tools the model calls in conversation.
<code>commands</code>	CommandDef[]	Slash commands. See flows .
<code>seeds</code>	object	Knowledge / glossary to add when enabled.
<code>settingsSchema</code>	object	Declares a per-server settings pane.
<code>components</code>	object	Persistent button/select handlers — see Persistent buttons .
<code>events</code>	object	Gateway-event hooks (e.g. <code>memberJoin</code>) — see Event hooks below. First-party only.
<code>onEnable</code>	function	Runs once on the OFF → ON transition (durable setup).

Tools

A **tool** is a function the language model can call on its own while your extension is enabled — this is how Olisar "looks things up" mid-conversation. Each tool declares a name, a description (the model reads this to decide when to use it), a JSON-schema for its arguments, and a handler that **returns a short string** for the model to weave into its reply.

```
` { name: "weather", description: "Current weather for a city.", parameters: { type: "object",
properties: { city: { type: "string", description: "city name" } }, required: ["city"], },
handler: async (args, ctx) => { const r = await host.fetch("https://wttr.in/" +
encodeURIComponent(args.city) + "?format=3") return await r.text() }, } `
```

The handler's second argument, `ctx`, carries `guildId`, `channelId`, `userId`, and `displayName` for the current conversation. Keep the returned string short and factual — Olisar rephrases it in its own voice.

DEGRADE POLITELY

Return a friendly string on failure ("couldn't reach the weather service") rather than throwing — Olisar will pass it along in character. Uncaught errors become a generic tool-failed message.

host capabilities

Each `host` method works **only if you listed its permission**. Calling one you didn't request throws.

Capability	Permission	What it does	
<code>host.fetch(url, init?)</code>	<code>fetch</code>	Public HTTP(S) only. Caps size/time/calls. <code>init.bodyBlobId</code> sends a host blob as the raw body; <code>init.responseBlob: true</code> stores the response as <code>res.blobId</code> ($\leq \sim 25$ MB). Otherwise returns text via <code>text()</code> / <code>json()</code> ($\leq \sim 20$ MB).	
<code>host.secret(ref)</code>	<code>secret:<ref></code>	Read an operator-approved key by reference (e.g. <code>host.secret("uex_api_key")</code>). You never see the literal value while authoring.	
<code>host.kb.addSource(seed)</code>	<code>kb.write</code>	Add a URL/website to the server's knowledge base. Idempotent.	
<code>host.glossary.add(fact)</code>	<code>glossary.write</code>	Add a <code>{ subject, fact }</code> to the glossary.	
<code>host.kv.get/set/delete</code>	<code>kv</code>	A small per-server key/value store your extension owns.	
<code>host.settings.get(key?)</code>	—	Read what an admin typed in your settings pane. No permission needed.	
<code>host.embed(spec)</code>	—	Build a Discord embed to pass to <code>reply({ embed })</code> .	
<code>host.log(msg)</code>	—	Write a line to the bot log. Always available.	
<code>host.files.ingest(optionName)</code>	—	Load a slash <code>attachment</code> into a host blob (<code>{ blobId, filename, size }</code>). Prefer for large files / API pipelines — bytes never enter the sandbox ($\leq \sim 25$ MB).	
<code>host.files.read(optionName)</code>	—	Load a slash <code>attachment</code> as base64 into the sandbox (<code>{ contentB64, ... }</code> , $\leq \sim 20$ MB). Fine for small files.	
<code>`host.files.from({ name, text\`</code>	<code>contentB64 }}</code>	—	Create a host blob from sandbox data.

Capability	Permission	What it does
<code>host.discord.*</code> (via the interaction)	<code>discord.reply</code> , <code>discord.modal</code> , <code>discord.components</code>	Reply (optionally with <code>files</code> / <code>blobId</code>), pop forms, and use buttons — see flows .
<code>host.generate({ task, maxTokens? })</code>	<code>model.generate</code>	Generate text in your server's persona voice (the persona is applied as the system prompt for you). Resolves to a string. First-party only .
<code>host.discord.send(channelId, payload)</code>	<code>discord.send</code>	Post a message to a channel (content / embed / components / files) — for event hooks that have no interaction to reply to.

HOST SECRETS AND SHARED CODE

`host.secret` reads the **operator's** keys (Gemini, Cloudflare, UEX). That's fine for extensions you wrote yourself, but extensions **installed from a file or the marketplace are blocked from host secrets entirely** — see [Security & trust](#). If you're publishing, don't rely on `host.secret`.

Seeds and onEnable

`seeds` lets a code-free (or any) extension contribute knowledge and glossary the moment it's switched on, applied idempotently:

```
`` seeds: { kbSources: [{ type: "url", uri: "https://example.com/faq", title: "Project FAQ" }],
glossary: [{ subject: "HQ", fact: "Coordination happens in #command." }], } ``
```

For anything more involved, `onEnable(ctx)` runs once on the OFF → ON transition (`ctx.guildId` tells you which server) — use it to seed durable state with `host.kv` or `host.kb`.

A settings pane

Declare `settingsSchema` and Olisar renders a config form on the extension's detail panel; read what the admin entered with `host.settings.get()`:

```
`` settingsSchema: { fields: [ { key: "channel", type: "channel", label: "Announcement channel"
}, { key: "intro", type: "textarea", label: "Intro message" }, ] } // later, in a handler:
const cfg = await host.settings.get() // { channel, intro } ``
```

Field types: `text`, `textarea`, `channel`, `number`, `toggle`. Settings are **per server**, so each server configures the extension its own way.

Event hooks

An extension can react to Discord **gateway events** by declaring an `events` map. The host runs your handler in the sandbox when the event fires for a server where your extension is enabled. The event you can hook is

`memberJoin`:

```
`` permissions: ["model.generate", "discord.send"], settingsSchema: { fields: [{ key:
"channel_id", type: "channel", label: "Welcome channel" } ] }, events: { async memberJoin(ctx)
{ const cfg = await host.settings.get() if (!cfg.channel_id) return const text = await
```

```
host.generate({ task: "Welcome " + ctx.member.displayName + " to the server in one warm sentence.", maxTokens: 200, }) await host.discord.send(cfg.channel_id, ctx.member.mention + " + text) }, } ``
```

The handler's `ctx` carries `guildId` and `member` (`{ id, displayName, username, mention, bot }`). There's **no interaction to reply to** — an event handler posts with `host.discord.send(channelId, payload)`, and can generate a message in the server's voice with `host.generate(...)`.

FIRST-PARTY ONLY

Event hooks, `host.generate`, and `host.discord.send` run only for **built-in and locally-authored** extensions — never imported or marketplace code, the same bar as [host secrets](#). The built-in **Welcome** extension is the worked example (open *Edit code* on it).

systemNote

A short instruction folded into Olisar's system prompt while the extension is enabled — use it to tell Olisar when to reach for your tools, or how to behave. Keep it brief; it's always in context.

Slash commands & flows

Extensions can add **slash commands** — including multi-step flows with pop-up forms and buttons. Commands re-register with Discord automatically when you save (and when an admin toggles the extension).

Defining a command

```
` defineExtension({ id: "poll", name: "Poll", permissions: ["discord.reply"], commands: [{  
  name: "ping", description: "Check that Olisar is alive.", handler: async (i) => { await  
    i.reply("pong") }, }, ], }) `
```

Command field	Type	Notes
<code>name</code> / <code>description</code>	string	As they appear in Discord's slash-command list.
<code>options</code>	OptionDef[]	Inputs: { <code>name</code> , <code>description</code> , <code>type</code> , <code>required</code> }. Types: <code>string</code> , <code>integer</code> , <code>number</code> , <code>boolean</code> , <code>user</code> , <code>channel</code> , <code>attachment</code> .
<code>defaultMemberPermissions</code>	string or null	<code>"manage_guild"</code> to limit it to server managers, or <code>null</code> for everyone.
<code>guildOnly</code>	boolean	Disallow the command in DMs.
<code>handler(i)</code>	function	Runs the command; <code>i</code> is the live interaction.

Read option values from `i.options`:

```
` commands: [{ name: "echo", description: "Repeat a message.", options: [{ name: "text",  
  description: "what to say", type: "string", required: true }], handler: async (i) => { await  
  i.reply(i.options.text) }, }, ] `
```

File uploads (attachment options)

Use `type: "attachment"` so Discord shows a file picker. `i.options.<name>` is **metadata only** ({ `id`, `filename`, `size`, `contentType` }) — no filesystem path. Load the file two ways:

Method	What you get	Size cap	When to use
<code>host.files.ingest(name)</code>	{ <code>blobId</code> , <code>filename</code> , <code>size</code> } (bytes stay on the host)	~25 MB	Large files, external APIs, reply with the result
<code>host.files.read(name)</code>	{ <code>contentB64</code> , ... } (base64 into the sandbox)	~20 MB	Small files you process in JS

Preferred pipeline (upload → external API → reply with file) — nothing large enters QuickJS:

```
` permissions: ["discord.reply", "fetch"], commands: [{ name: "compress", description:  
  "Compress a file via an external API.", options: [{ name: "file", description: "File to  
  compress", type: "attachment", required: true }], handler: async (i) => { await i.reply({  
    content: "Compressing...", ephemeral: true }) // ack within 3s const input = await  
    host.files.ingest("file") // host blob const res = await
```

```
host.fetch("https://api.example.com/compress", { method: "POST", headers: { "Content-Type":
"application/octet-stream" }, bodyBlobId: input.blobId, // raw bytes as the request body
responseBlob: true, // store response as another host blob }) if (!res.ok || !res.blobId) {
await i.followUp({ content: "Compression failed.", ephemeral: true }) return } await
i.followUp({ content: "Done – compressed *" + input.filename + ".*", files: [{ name:
input.filename + ".gz", blobId: res.blobId }], // stays private: followUps inherit ephemeral
from the first reply }) }, }]
```

Small-file path with base64 still works:

```
` const file = await host.files.read("doc") // { filename, size, contentType, contentB64 }
await i.reply({ content: "Got " + file.filename + " (" + file.size + " bytes)", ephemeral:
true }) `
```

File output (attach on reply)

`reply` / `followUp` / `host.discord.send` accept `files`. Each entry needs one of:

- `text` – UTF-8 ($\leq \sim 20$ MB)
- `contentB64` – binary as base64 ($\leq \sim 20$ MB)
- `blobId` – host-held blob from `ingest` / `from` / `fetch({ responseBlob: true })` ($\leq \sim 25$ MB)

Max 10 files; **total** size capped at ~ 25 MB (Discord's bot limit):

```
` await i.reply({ content: "Here's your export.", files: [ { name: "report.csv", text:
"a,b\n1,2\n" }, { name: "out.bin", blobId: someBlob.blobId }, ], }) `
```

The interaction object

The handler's `i` exposes the conversation context (`guildId`, `channelId`, `userId`, `displayName`) and:

- `i.reply(payload)` – the first response. A string, or `{ content, embed, ephemeral, components, files }`.
- `i.followUp(payload)` – additional messages after the first (same shape; supports `ephemeral` and `files`).
- `i.modal(spec)` – pop a form and **await** the submitted values (permission `discord.modal`).
- `i.awaitComponent({ timeoutMs })` – wait for a button click / menu choice (permission `discord.components`).

`reply` / `followUp` need `discord.reply`. Use `ephemeral: true` to make a message visible only to the caller.

If the **first** reply is ephemeral, later `followUp`s stay private by default (you can still set `ephemeral: false` to post publicly). Use `host.files.read` / `ingest` for attachment option bytes (command handlers only).

A form (modal)

`i.modal` opens a Discord form and resolves with the submitted fields, keyed by `id`:

```
` permissions: ["discord.reply", "discord.modal"], commands: [{ name: "suggest", description:
"Submit a suggestion.", handler: async (i) => { const f = await i.modal({ title: "New
suggestion", fields: [ { id: "title", label: "Title", style: "short", required: true }, { id:
"body", label: "Details", style: "paragraph" }, ], }) await i.reply({ content: "Thanks!
Logged: " + f.title, ephemeral: true }) }, }]
```

Buttons and menus

Send components with `reply`, then wait for the interaction:

```
` permissions: ["discord.reply", "discord.components"], handler: async (i) => { await
i.reply({ content: "Ready to launch?", components: [ { kind: "button", customId: "go", label:
"Launch", style: "primary" }, { kind: "button", customId: "cancel", label: "Cancel", style:
"secondary" }, ], }) const c = await i.awaitComponent({ timeoutMs: 30000 }) await
i.followUp(c.customId === "go" ? "Launching!" : "Cancelled.") } `
```

A `select` component returns the chosen values in `c.values`. If nobody responds within `timeoutMs`, the `await` rejects — catch it and tidy up.

Persistent buttons

`awaitComponent` is for a single, short-lived prompt — it stops listening after `timeoutMs` (and after a bot restart). For buttons many people click over hours or days — polls, RSVPs — declare a ``components`` map instead. Each handler has a short key; a button/select that references it by `handlerId` keeps working for everyone and **survives restarts** (no per-message rebuild).

```
` permissions: ["kv", "discord.reply", "discord.components"], components: { // keyed by
handlerId; runs on every click, by anyone, forever vote: async (i) => { const tally = (await
host.kv.get("tally")) || {} tally[i.userId] = i.arg // i.arg is the small payload you set on
the button await host.kv.set("tally", tally) await i.update({ embed: host.embed({ title:
"Votes: " + Object.keys(tally).length }) }) }, }, commands: [{ name: "poll", description:
"Start a poll.", handler: async (i) => { await i.reply({ content: "Pick one:", components: [ {
kind: "button", handlerId: "vote", arg: "a", label: "A" }, { kind: "button", handlerId:
"vote", arg: "b", label: "B" }, ], }) } ] } `
```

A persistent handler receives a **ComponentInteraction** `i` with `customId`, `arg`, `values` (for selects), and the usual context. Its methods differ from a command's:

- `i.reply(payload)` — answer the **clicker privately** (ephemeral).
- `i.update(payload)` — edit the **source message** in place (live tally, attendee list). Pass `components: []` to clear the buttons; omit `components` to leave them.
- `i.deferUpdate()` — acknowledge the click with no visible change.

Keep `handlerId` + `arg` short (under ~40 chars); store anything bigger in `host.kv` and pass its key as `arg`. The host stamps the routing id, so a click can only ever reach the extension that owns it.

Embeds

Build rich cards with `host.embed` and pass them to `reply`:

```
` const card = host.embed({ title: "Status", description: "All systems nominal.", color:
0x2e9fff, fields: [{ name: "Uptime", value: "5d 2h", inline: true }], footer: "live", }) await
i.reply({ embed: card }) `
```

Sharing extensions

Extensions move between bots as `.olx` files — a small, signed bundle. You can hand one to a friend directly, or use [the marketplace](#) (which is built on the same format).

Exporting

On any extension you can edit, the detail panel has an **Export** button. It downloads `<id>-<version>.olx` — a JSON document containing your extension's **source** (not compiled code), its metadata and declared permissions, an integrity hash, and a **signature** from your bot's publisher key.

Importing

The **Import .olx** button on the [Extensions](#) tab opens a file picker, then shows a **review screen** before anything is installed:

- **What it adds** — its tools and commands.
- **Signature** — **Signed & verified** (with the publisher's fingerprint), **Unsigned**, or **Signature invalid**.
- **Capabilities to grant** — every permission it requests, as checkboxes. You grant a subset; anything you leave unchecked simply won't work for the extension.

Press **Install** and it's added as a custom extension you can then enable per server.

THE BOT RE-DERIVES EVERYTHING

On import, Olisar **re-transpiles the source itself** and re-checks the signature — it never trusts pre-built code from a file. A bundle whose signature doesn't match its contents is refused outright.

IMPORTED CODE IS THIRD-PARTY

An imported extension runs real code in your bot. Grant only the capabilities you're comfortable with, and note that **host secrets are off-limits to imported extensions** regardless of what you grant (see [Security & trust](#)). Prefer extensions from a **verified publisher**.

Signing, briefly

Your bot has its own Ed25519 **publisher key**, created automatically the first time you export or publish. The private key never leaves your machine; the public key (and a short **fingerprint**) travel with your bundles so others can confirm a bundle is really from you and hasn't been altered. More in [Security & trust](#).

The marketplace

The marketplace is a shared catalog of extensions, hosted on Cloudflare. Browsing, installing, and publishing all happen from your console — the bot talks to the registry for you.

Browsing and installing

On the **Extensions** tab, **Marketplace** opens a searchable catalog. Each result shows the publisher (with a ✓ **verified** badge if they're Discord-verified), the version, and the capabilities it requests. **Install** runs the exact same **consent screen** as a file import — review what it adds and what it can access, grant a subset of permissions, and confirm. The bot downloads the bundle, re-verifies its signature, and installs it as a custom extension.

INSTALLED = THIRD-PARTY

Marketplace extensions are held to the same rules as file imports: re-verified on install, granted only the capabilities you approve, and **blocked from host secrets** (see **Security & trust**).

Publishing your own

On an extension you authored, the detail panel has a **Publish** button. The first time, you'll be asked to **claim a publisher handle** — your namespace in the catalog (e.g. `m-studio`). It's bound to your bot's **publisher key**: once you own a handle, only your key can publish under it, and every bundle you publish is signed by it.

Once it's live, the panel shows a **Published** badge with its catalog version. Edit the code and it flags **unpublished changes**, and the button becomes **Push update** — click it to publish your new source. Bump the `version` in your code first if you want existing installs to be **offered the update**: a same-version re-publish overwrites in place, so people who already installed it won't be prompted to update.

Removing a version

Yank pulls a version — or the whole extension — from the catalog. It stops appearing for everyone; anyone who already installed it sees a **Removed from marketplace** note but it keeps working. If an extension you **installed** from the marketplace is later yanked, it automatically **reverts to a plain local extension** — it drops the Marketplace label, keeps the capabilities you granted, and becomes publishable again, so you can keep using it or re-list it under your own handle.

The verified badge

Claiming a handle proves you hold the key; the **verified** badge additionally proves the handle belongs to a real Discord account. In the Marketplace view, a registered publisher sees **Verify with Discord** — click it, approve on Discord, and your published extensions show a ✓ **Discord-verified** badge to everyone.

ONE-TIME SETUP FOR VERIFICATION

Verification uses a Discord OAuth redirect, so you must register its callback URL in your bot's Discord app (Developer Portal → your app → **OAuth2** → **Redirects**), next to your existing login redirect: `<your console URL>/api/marketplace/verify/callback` (e.g. `http://localhost:8000/api/marketplace/verify/callback`). Without it, Discord rejects the flow with `invalid redirect_uri`. See **Hosting & access** for your URL.

Self-hosting / pointing elsewhere

The registry the console uses is configurable via the `OLISAR_REGISTRY_URL` environment variable (it defaults to the official hosted one). Point it at your own Cloudflare Worker to run a private marketplace — the bundle format and signing are the same, so trust still travels with each signed `.olx`.

COST

The hosted registry runs within Cloudflare's free tier, with hard caps on storage and writes so it can never bill. Bundles are tiny (source only), so a catalog is effectively free to run.

Security & trust

Extensions run real code, so Olisar runs them under a strict, layered security model. This page explains what protects you — useful whether you're authoring, installing, or just deciding whether to trust an extension.

The sandbox

Every extension runs in a **hermetic JavaScript sandbox** with **no ambient authority**. It cannot touch the filesystem, open arbitrary network connections, read environment variables, or reach the bot's internals. The only way out is the `host.*` capabilities — and each of those works only if the operator granted its permission. Each run is bounded by **CPU, memory, and wall-clock limits**, so a slow or runaway extension can't hang the bot.

`host.fetch` is the one network door, and it's guarded: only public HTTP(S) hosts (loopback and private addresses are blocked, preventing access to internal services), with caps on response size, timeout, and the number of calls per run.

Permissions: requested vs granted

Two separate things:

- **Requested** — the capabilities an extension declares in `permissions`.
- **Granted** — what the operator actually approves.

When you author an extension you grant what you declare. When you **install** one from a file or the marketplace, the **consent screen** lets you grant a **subset** — uncheck anything you don't want, and that capability is simply unavailable to the extension at runtime.

Host secrets are off-limits to third parties

`host.secret` exposes the operator's own keys (Gemini, Cloudflare, UEX). **First-party** extensions (built-ins and ones you authored locally) may use them once granted. **Imported and marketplace** extensions are **blocked from host secrets entirely** — even if you tick the box — so installed third-party code can never read or exfiltrate your keys. The consent screen marks those requests as unavailable.

The same first-party bar applies to the other powerful capabilities: `host.generate` (host-paid model calls), `host.discord.send` (unprompted channel posts), and **event hooks** like `memberJoin`. Built-in and locally-authored extensions can use them; imported and marketplace code can't, regardless of what's granted.

Signing and integrity

Every bundle carries a **content hash** and an **Ed25519 signature**:

- The hash detects accidental corruption.
- The signature ties the bundle to a publisher's key, so it can't be tampered with or impersonated. Your bot's private key never leaves your machine; only the public key + a short **fingerprint** travel with bundles.

On install, Olisar re-derives the hash from the source and verifies the signature. **Valid** shows the publisher fingerprint; **Unsigned** means authorship can't be confirmed; **Invalid** blocks the install outright (the file was altered after signing).

On the marketplace, a **handle is owned by the key that first claimed it**, so only that key can publish under it. A ✓ **verified** publisher has additionally proven the handle maps to a real Discord account.

What's withheld

Even with every permission granted, extensions never get: the raw database or bot internals, the filesystem, arbitrary environment/secret values, the ability to DM arbitrary users, or `eval` /dynamic code loading. New capabilities are added deliberately, behind named permissions.

Trusting an installed extension

A quick checklist before installing third-party code:

- Prefer a ✓ **verified** publisher, or a bundle whose signature shows **Signed & verified**.
- Read the **capabilities** it asks for — does a dice roller really need `fetch`?
- Grant the **minimum** that makes it work; you can leave capabilities unchecked.
- Remember it can't reach your **host secrets** or anything outside the sandbox no matter what.

Privacy & data

Olisar is built to respect members' data, and to be transparent about what it keeps.

STORED LOCALLY, ON THE OPERATOR'S MACHINE

All of the below lives in a single database on the **operator's own computer** — there's no Olisar cloud (see [Hosting & your data](#)). Admins who sign in, locally or over [remote access](#), read and write that machine's data live.

What it stores

- **Messages** from channels set to `memory` / `both` (for conversation context), and a copy of **every** message in a separate **search index** (the all-channel index — an admin's explicit choice).
- **Summaries** of past conversation, a private **profile** of each member built from their messages, and **facts** it's chosen to remember.
- Short **descriptions of posted images**, and **embed/file** text, so they're searchable.
- **Reminders** you ask it to set — kept only until they're delivered, then marked done.

What it doesn't do

- It never shares DMs or private content publicly.
- **Your DMs stay in your DMs.** A 1:1 conversation with Olisar can only be recalled inside that same conversation — never in a channel, and never by another member. **Server admins are no exception:** Manage Server governs the server's own channels, not anyone's private messages.
- Opted-out members are **never** recorded or indexed.
- It treats recalled memory as background **data**, not as instructions it must obey.
- **Presence & voice** (what someone's playing, who's in voice) are read **live, only when a tool asks** and only if an admin turned on Status & voice awareness — they're never stored.
- The dashboard **Test chat** is memory-free: nothing said there is saved or mined.

Member controls

- `/privacy` — a plain-language summary of all of the above, available to anyone.
- `/forget-me` — deletes everything stored about a person: messages, facts, profile, and their entries in the search index. `stop_remembering: true` also opts them out of future recording, permanently. When a message is edited or deleted in Discord, Olisar updates or removes its copy too.

ADMIN WIPE

`/self-destruct` erases everything Olisar has **learned** across the whole server — memory, profiles, facts, the search index, and the knowledge base — while keeping its personality and your settings. It's irreversible, and members' opt-out choices survive it.

TIP

The all-channel search index is the one thing worth telling your members about up front. The `/privacy` text discloses it, and you can reword that text on the [Command replies](#) tab.

Troubleshooting & FAQ

Most issues come down to free-tier rate limits or a channel/access setting. Here's the quick reference:

Symptom	Likely cause	Fix
"My mind went blank" / slow replies	Free-tier rate limiting — every model busy at once	Wait a minute; on Behavior , start the chain at a less-contended model
Won't reply in a channel	Channel mode is <code>off</code> or <code>memory</code>	Set <code>respond</code> or <code>both</code> on Channels (threads/forum posts inherit the parent)
A member can't use it	A role is marked Allowed , locking everyone else out	Adjust the Access tab
Image generation fails	Cloudflare not configured, or the daily allocation is used up	Add the Cloudflare keys; otherwise wait for the daily reset
KB answers missing right after adding a site	Ingestion + embedding runs in the background, throttled	Give it time; check <code>/olisar sources</code> for status
Search can't find old messages	Only live messages are indexed going forward	Run <code>/olisar reindex</code> to backfill history
<code>/citizen</code> says the extension is off	Star Citizen extension disabled	Enable it on the Extensions tab
Web lookups stopped working	The daily web-search cap is used up	Raise it on Behavior , or wait for the reset
Olisar quoted a deleted message	Rare timing between the edit/delete and the sync	It syncs automatically — try again
Dashboard won't load / bot offline	The operator's machine is asleep, off, or Olisar was quit from the tray	Wake the machine and reopen Olisar — it must stay running (Hosting)
Other admins can't open the web link	Remote access is off, or the address changed	The operator turns it back on under Settings → Remote access and re-shares the link from the sidebar (Remote access)
Discord login bounces or says "invalid or expired state"	The redirect URL for that address isn't registered	Register the exact <code>.../auth/callback</code> the wizard shows (both the local and <code>...ts.net</code> ones)
A setting didn't take effect	The change is still buffered in the save bar	Press Save in the bar at the bottom of the page

STILL STUCK?

Check the [Usage](#) tab to see whether you're hammering the quota, then the bot's logs under **Settings** → **Logs** — they name the knowledge-base chunks, indexed messages, web sources, and tools behind each reply.